

گزارش پروژه کامپایلر - فاز سوم

در ادامه‌ی پروژه اصول طراحی کامپایلر، کدی که ارائه شده است ترکیبی از تحلیل نحوی (Parser) و تحلیل معنایی (Semantic Analyzer) می‌باشد. همان‌طور که در فایل پروژه‌ی فاز ۳ آمده است، هدف اصلی در این مرحله بررسی صحت معنایی برنامه است؛ یعنی اطمینان از این که دستورات نوشته شده علاوه بر درست بودن از نظر نحوی، از نظر معنایی نیز سازگار و قابل اجرا باشند. این شامل بررسی نوع داده‌ها، قوانین اسکوپ، تعریف و استفاده از متغیرها و توابع، و مدیریت خطاهای معنایی می‌شود.

فایل ورودی in.Java است. در انتها تصویر ورودی و خروجی متناسب با آن طبق خواسته پروژه، موجود است.

ویژگی‌های اصلی پیاده‌سازی

۱. جدول نماد (Symbol Table) و مدیریت اسکوپ

- از یک پشته از Map ها استفاده شده است تا اسکوپ‌های تو در تو مدیریت شوند.
- هر متغیر به صورت یک شیء Sym ذخیره می‌شود که شامل نام، نوع داده، شماره خط و وضعیت استفاده (used) است.
- هنگام ورود به بلوک جدید (enterScope) یک Map تازه به پشته اضافه می‌شود و هنگام خروج (exitScope) متغیرهای آن بررسی می‌شوند تا در صورت عدم استفاده هشدار داده شود:

```
125  
126 Deque<Map<String, Sym>> scopes = new ArrayDeque<>();  
127
```

۲. بررسی تعریف و استفاده از متغیرها

- تابع `declareVar` وظیفه‌ی ثبت متغیر جدید را دارد و در صورت تعریف دوباره یا **Shadowing** هشدار/خطا چاپ می‌کند.
- تابع `useVar` بررسی می‌کند که متغیر قبل از استفاده تعریف شده باشد. در غیر این صورت خطای `"[Error] variable used before declaration"` داده می‌شود.

```
void exitScope() {  
    Map<String, Sym> scope = scopes.pop();  
    for (Sym s : scope.values()) {  
        if (!s.used) {  
            System.out.println("[Warning] variable '" + s.name +  
                "' declared at line " + s.line + " but never used.");  
        }  
    }  
}
```

۳. بررسی سازگاری نوع داده‌ها

- برای عملیات ریاضی (جمع، تفریق، ضرب، تقسیم، مدول) توابع جداگانه مانند `addNumbers` و `subNumbers` پیاده‌سازی شده است.
- در تولید نوع داده‌ی نهایی محاسبه می‌شود و در صورت ناسازگاری هشدار داده می‌شود:

```
1588  ~ if (!isNumericType(left) || !isNumericType(right)) {  
1589  ~     System.out.println("[Error] Type mismatch: cannot add non-numeric types at line " + lineNum);  
1590  ~ }  
1591  ~
```

- در دستورات انتساب (assignment) نیز نوع متغیر سمت چپ با نوع عبارت سمت راست مقایسه می‌شود و خطای ناسازگاری چاپ می‌شود.

۴. تحلیل توابع

- برای هر تابع یک شیء FunctionInfo ایجاد می‌شود که شامل نام، نوع بازگشتی و لیست پارامترهاست.
- در زمان فراخوانی تابع، تعداد و نوع آرگومان‌ها بررسی می‌شود:

```

1453         if (argList.size() != f.parameters.size()) {
1454             System.out.println("[Error] Function '" + funcName + "' expects " +
1455                 f.parameters.size() + " args but got " + argList.size() + " at line " + idleft);
1456         } else {
1457             for (int i = 0; i < argList.size(); i++) {

```

در صورت ناسازگاری نوع پارامتر یا تعداد آن‌ها خطای معنایی چاپ می‌شود.

۵. بررسی دستور return

- با استفاده از متغیر currentFunction نوع بازگشتی تابع پیگیری می‌شود.
- در صورت نبودن مقدار بازگشتی در تابع غیر void یا برعکس، خطا تولید می‌شود:

```

1320 return_statement ::=
1321     RETURN expression_opt:exp SEMICOLON
1322     {:
1323         if (currentFunction != null) {
1324             if ("void".equals(currentFunction.returnType)) {
1325                 if (exp != null) {
1326                     int lineNum = ((java_cup.runtime.Symbol)CUP$parser$stack.elementAt(CUP$parser$stop-1)).left;
1327                     System.out.println("[Error] Cannot return value from void function '" +
1328                         currentFunction.name + "' at line " + lineNum);
1329                 }
1330             } else {
1331                 if (exp == null) {
1332                     int lineNum = ((java_cup.runtime.Symbol)CUP$parser$stack.elementAt(CUP$parser$stop-1)).left;
1333                     System.out.println("[Error] Missing return value in function '" +
1334                         currentFunction.name + "' at line " + lineNum);
1335                 } else {
1336                     String exprType = getExpressionType(exp);
1337                     if (exprType != null && !currentFunction.returnType.equals(exprType)) {
1338                         int lineNum = ((java_cup.runtime.Symbol)CUP$parser$stack.elementAt(CUP$parser$stop-1)).left;
1339                         System.out.println("[Error] Return type mismatch in function '" +
1340                             currentFunction.name + "': expected " + currentFunction.returnType +
1341                             " but got " + exprType + " at line " + lineNum);
1342                     }
1343                 }
1344             }
1345         }
1346     }
1347

```

۶. پشتیبانی از ساختارهای کنترلی

- دستورات شرطی (if, else if, else) و حلقه‌ها (for, while, do while) شناسایی و در خروجی گزارش می‌شوند.

۷. تولید (AST)

- کلاس‌هایی مثل BinaryOpNode, VariableNode, NumberNode برای ساخت AST پیاده‌سازی شده‌اند.
- در عملیات ریاضی یک گره دودویی ساخته و چاپ می‌شود.

خطاها و هشدارهای معنایی

نمونه‌هایی از خطاها و هشدارهایی که کد گزارش می‌دهد:

- استفاده از متغیر قبل از تعریف.
- تعریف دوباره‌ی متغیر در یک اسکوپ.
- Shadowing متغیرها در اسکوپ داخلی.
- ناسازگاری نوع در انتساب.
- خطا در تعداد یا نوع آرگومان‌های تابع.
- بازگشت مقدار در تابع void یا عدم بازگشت مقدار در تابع غیر-void.
- متغیر تعریف شده اما استفاده نشده.

ورودی / خروجی

```
class Test {  
    public static int main() {  
        int x;  
        int y = 10;  
        x = 5 + 2 * 3;  
  
        undeclaredFunc();  
        return x ;  
    }  
  
    public static String testFunc(int a, float b) { return 1; }  
}
```

JavaParser ×

```
"C:\Program Files\Java\jdk-19\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA\lib\idea_rt.jar" 123456789  
Parsing [in.java]  
function Variable: x, Initial Value: null  
function Variable: y, Initial Value: 10  
AST: (5 + 6)  
[Error] Call to undeclared function 'undeclaredFunc' at line -1  
[Warning] variable 'y' declared at line -1 but never used.  
FunctionDef:Function: main, Return type: int, Parameters: none  
FunctionDef:Function: testFunc, Return type: String, Parameters: int a, float b  
ClassDef:Test  
No errors.
```